Pipeline Integrado

1. Introdução

O arquivo `main.py` representa o **núcleo do sistema**, responsável por integrar todas as etapas desenvolvidas anteriormente em um único pipeline automatizado.

Ele realiza, de forma contínua:

- leitura de imagens de veículos detectados;
- detecção da placa com YOLO;
- pré-processamento da imagem da placa;
- leitura OCR com EasyOCR;
- correção heurística (padrão Mercosul);
- registro dos dados em planilha;
- organização e arquivamento dos dados processados.

Esse módulo transforma o projeto em um sistema **autônomo e funcional para uso real**.

2. Arquitetura do Pipeline

O fluxo implementado segue a seguinte sequência:

```
Tracking → Recorte do carro → YOLO (placa) → Pré-processamento → OCR → Correção → Registro → Armazenamento
```

3. Estrutura de Diretórios

O sistema utiliza três diretórios principais:

```
DIR_ENTRADA="fotos_carros"
DIR_PRONTAS="placas_finalizadas"
DIR_HISTORICO="historico_carros"
```

Funções:

- **fotos_carros/** → entrada (imagens vindas do tracking)
- **placas_finalizadas/** → placas tratadas e prontas para OCR
- **historico_carros/** → backup das imagens já processadas

4. Carregamento dos Modelos

```
yolo_placa=YOLO("runs/detect/placa_detector/weights/best.pt")
reader=easyocr.Reader(["en"],gpu=True)
```

Justificativa:

- YOLO especializado → detecta apenas placas
- EasyOCR → leitura robusta de caracteres
- GPU ativada → melhora desempenho em tempo real

5. Correção Heurística (Mercosul)

A função `corrigir_mercosul()` corrige erros comuns do OCR:

Estratégia:

- remove caracteres inválidos;
- aplica substituições inteligentes (ex: O ↔ 0);
- respeita o padrão:

```
LLL N L NN
```

Benefício:

- reduz erros típicos de OCR;
- aumenta a confiabilidade do sistema;
- padroniza a saída final.

6. Pré-processamento Integrado

A função `preprocess()` aplica:

Etapas:

1. **Remoção da faixa superior da placa**
   - elimina ruído do padrão Mercosul ("Brasil")
2. **Redimensionamento**
   - largura fixa de 600px
   - melhora leitura do OCR
3. **Conversão para escala de cinza**
4. **CLAHE**
   - aumenta contraste local
5. **Binarização de Otsu**
   - separa texto e fundo
6. **Sharpening**
   - melhora contornos dos caracteres

Resultado:

- imagem limpa, nítida e ideal para OCR

7. Processamento Contínuo (Loop Principal)

```
whileTrue:
```

O sistema opera em modo contínuo, realizando:

Etapas:

1. verifica novas imagens na pasta de entrada;
2. processa cada imagem individualmente;
3. aplica detecção da placa;
4. executa OCR e correção;
5. registra os dados;
6. move o arquivo para histórico.

Característica:

- sistema **quase em tempo real**
- tolerante à ausência de arquivos (uso de `sleep`)

8. Detecção da Placa no Veículo

```
res=yolo_placa(img_carro)
```

- extrai bounding box da placa;
- recorta automaticamente a região;

Benefício:

- elimina necessidade de detecção manual;
- garante integração com o tracking.

9. Leitura OCR

```
ocr_res=reader.readtext(...)
```

Fluxo:

1. leitura da imagem tratada;
2. extração do texto bruto;
3. limpeza e correção;
4. cálculo da confiança.

Saídas:

- texto da placa
- nível de confiança

10. Registro dos Dados (Excel)

Os dados são armazenados em:

```
registro_placas.xlsx
```

Campos registrados:

- Data_Hora
- Arquivo_Original
- ID_Veiculo
- Texto_Placa
- Confianca
- Path_Placa_Limpa

Lógica:

- cria arquivo se não existir;
- concatena novos registros ao existente;

Benefícios:

- persistência dos dados;
- fácil integração com sistemas administrativos;

- suporte à auditoria.

11. Organização dos Arquivos

Após processamento:

```
shutil.move(...)
```

- imagens são movidas para histórico;
- evita reprocessamento;
- mantém organização do sistema.

12. Resiliência do Sistema

O sistema inclui:

- verificação de arquivos disponíveis;
- tratamento de erros no Excel;
- fallback em falhas de leitura OCR;
- loop contínuo com pausas controladas.

13. Fluxo Completo do Sistema

1. Tracking salva imagem do carro
2. `main.py` detecta nova imagem
3. YOLO encontra a placa
4. Imagem é pré-processada
5. EasyOCR lê o texto
6. Texto é corrigido
7. Resultado é salvo em Excel
8. Imagem é movida para histórico

14. Justificativa da Abordagem

A centralização em um único script foi adotada porque:

- simplifica execução do sistema;
- facilita integração entre módulos;
- reduz dependências externas;
- permite operação contínua automatizada.

Além disso:

- o uso de heurísticas melhora o OCR;
- o armazenamento estruturado permite análise futura;
- o sistema torna-se aplicável em cenários reais (portarias, estacionamentos, vigilância).

15. Conclusão

O `main.py` consolida todo o pipeline desenvolvido, transformando o projeto em uma solução completa de **Reconhecimento Automático de Placas Veiculares (ANPR)**.

A integração entre detecção, processamento, OCR e armazenamento garante um sistema robusto, escalável e adequado para aplicações reais de monitoramento e controle de acesso.